Carrera: Ingeniería en Sistemas Computacionales

Materia: Tópicos de inteligencia artificial

Título: optimización de rutas mediante la búsqueda tabú

Alumnos:

Jesús Eduardo Lizárraga Valenzuela

Guillermo Eduardo padilla Noriega Padilla

Miguel Matzumoto Urquiza

Fecha: 30/03/2026

optimización de rutas mediante la búsqueda tabú

Introducción

Cuando se habla de reducir costos en el sector de distribución la eficiencia logística es la clave para hacerlo En esto proyecto aplica inteligencia artificial para resolver este problema de logística en la ciudad de Culiacán Sinaloa, tenemos que lograr mover productos de los centros de distribución hasta las sucursales utilizando la ruta mas eficiente

El problema principal es que tenemos 90 puntos de entrega lo que hace que tengamos muchas rutas diferentes posibles. Por esto implementaremos la búsqueda tabú, para encontrar las rutas con mas bajo costo de manera eficiente, nuestro objetivo es desarrollar una solución computacional que minimice la distancia y el gasto de combustible, mejorando gasto y la rentabilidad.

Objetivo especifico

el objetivo general del proyecto es diseñar e implementar un solución computacional basada en el algoritmo de búsqueda tabu para a si encontrar las rutas optimas de distribución de los abarrotes en Culiacán, con el fin de mejorar la eficiencia logística y reducir los costos operativos, para lograrlo primero que nada tenemos que hacer un análisis detallado del problema logístico y la red de sucursales, crear un modelo matemático para el sistema como un caso de optimización combinatoria y la programación de algoritmos en Python en la cual utilizaremos las matrices de distancia y combustible.

Justificación

Para armar rutas para 100 puntos de entregas en una ciudad tan grande como la de Culiacán podría ser muy complicado, si quisiéramos probar todas las combinaciones posibles para ver cual es la mejor tomaría mucho tiempo porque las opciones son casi infinitas, por esto es que en este proyecto decidimos usar métodos de búsqueda heurísticos

A diferencia de andar adivinado cual es la mejor ruta, la búsqueda tabu nos permite encontrar la ruta más eficiente en mucho menos tiempo, ahorrando tiempo y dinero

Alcance

en este proyecto nos enfocamos encontrar la mejor ruta para los 100 puntos de entrega, nuestro sistema calculara el camino que sea mas optimo o barato basándose en la distancia y el precio del combustible

lo que no se tomara en cuenta para elaborar nuestro proyecto será cosas que cambian al momento como el tráfico, accidentes, clima, tipo de terreno por lo que pasara el conducto, ya que nuestro objetivo es encontrar la ruta optima únicamente

agenda de trabajo

Fase del proyecto	actividad	Duración	Fecha
Análisis del problema	Analizar detalladamente el problema a resolver	1 día	22/03/2026
Modelo matemático	Definir la función objetivo y el planteamiento de el problema	1 día	23/03/2026
Selección del algoritmo	Comparar entre el recocido simulado y búsqueda tabú y justificar el elegido	1 día	24/03/2026
Desarrollo del problema	Programación del código	3 dias	25/03/2026 - 28/03/2026
Pruebas y ajuste	Ejecutar el simulador para ajustar el tamaño de la lista tabú y el numero de interacciones	1 días	29/03/2026
Documentación	Redacción del reporte, conclusión y formato del documento	1 día	30/03/2026

Recocido simulado

El recocido simulado toma su nombre y principios fundamentales del proceso metalúrgico de recocido, donde los metales se calientan a altas temperaturas y luego se enfrían lentamente para reducir sus defectos y aumentar su resistencia. Desarrollado en los años 80 por Kirkpatrick, Gelatt y Vecchi, el algoritmo imita matemáticamente este proceso físico para encontrar soluciones óptimas en espacios de problemas complejos.

La idea básica del recocido simulado es que la aleatoriedad controlada, al igual que el movimiento aleatorio de los átomos durante el enfriamiento, puede ayudar a encontrar mejores soluciones a problemas de optimización. A medida que disminuye la “temperatura”, el sistema se asienta gradualmente en un estado de energía mínima, lo que en términos de optimización se traduce en encontrar una solución que minimice (o maximice) la función objetivo.

búsqueda tabú

búsqueda tabú (TS) es un método de búsqueda metaheurística que emplea métodos de búsqueda locales utilizados para la optimización matemática. Fue creado por Fred W. Glover en 1986 y formalizado en 1989.

Las búsquedas locales (de vecindario) toman una posible solución a un problema y verifican sus vecinos inmediatos (es decir, soluciones que son similares excepto por muy pocos detalles menores) con la esperanza de encontrar una solución mejorada. Los métodos de búsqueda local tienden a quedarse estancados en regiones subóptimas o en mesetas donde muchas soluciones son igualmente adecuadas.

La búsqueda tabú mejora el rendimiento de la búsqueda local al relajar su regla básica. En primer lugar, en cada paso se pueden aceptar movimientos *que empeoran* si no hay ningún movimiento de mejora disponible (como cuando la búsqueda está estancada en un mínimo local estricto). Además, se introducen *prohibiciones* (de ahí el término *tabu*) para desalentar la búsqueda de volver a soluciones visitadas anteriormente.

La implementación de la búsqueda tabú utiliza estructuras de memoria que describen las soluciones visitadas o los conjuntos de reglas proporcionados por el usuario. Si una solución potencial ha sido visitada previamente dentro de un período de corto plazo determinado o si ha violado una regla, se marca como "tabú" (prohibido) para que el algoritmo no considere esa posibilidad repetidamente.

Comparación

El recocido simulado y la búsqueda tabú son metaheurísticas utilizadas para resolver problemas de optimización.

El recocido simulado se basa en un proceso probabilístico este utiliza una temperatura que va disminuyendo con el tiempo permitiendo tener soluciones peores al inicio y a como avanza el tiempo se vuelve más estricto con las soluciones

La búsqueda tabú utiliza un enfoque basado en la memoria, se emplea una lista tabú que evita repetir soluciones ya visitadas lo que ayuda a encontrar nuevas soluciones.

Se podría decir que el recocido simulado se caracteriza por su aleatoriedad y exploración global mientras que la búsqueda destaca por el uso de memoria y el control de búsqueda

Algoritmo elegido para implementar en nuestro proyecto

Como ya vimos antes la búsqueda tabú destaca por su uso de memoria y control de búsqueda haciendo que evite repetir soluciones y visitadas por este motivo creemos que para este proyecto seria lo más adecuado utilizar la búsqueda tabú ya que con este método vamos descartando rutas que no sean efectivas para encontrar la ruta mas efectiva para cada centro de distribución

Desarrollo

Nosotros trabajamos utilizando como lenguaje de programación Python, ya que fue el más adecuado y necesario para el desarrollo del proyecto. Elegimos este lenguaje porque ofrece una gran facilidad para implementar funciones, estructuras y herramientas que ayudan a resolver problemas de manera más eficiente. Además, Python cuenta con una sintaxis clara y sencilla, lo que permite escribir y comprender el código con mayor facilidad en comparación con otros lenguajes

Librerias utilizadas:

Pandas: Utilizamos pandas para poder pasar el excel en matrices y sea mas facil la lectura ya que es muy complicado pasar todo el codigo manual y mas si es de 100x100, un total de 1000 celdas para la distancias y otras 1000 celdas para los costos de combustible

Random: Utilizamos el importe random porque nos facilita el generar una ruta inicial aleatoria al momento de querer realizar el método tabu haciendo mas comodo y mas realista los vecinos

Variables utilizadas:

archivo_distancias: Aqui se guardara la ruta de nuestro archivo excel de distancias

archivo_combustible: Aqui se guardara la ruta de nuestro archivo excel de costo combustible

centros: Aqui se creara una lista donde tomara un rango del 1 al 11 ya que en el excel de distancias asi estan visualizados los centros de distribucion y se guardaran automaticamente

tiendas: Igual que la variable centros esta es una lista pero tomara los rangos del 11 al 101 donde se guardaran automáticamente las tiendas

iteraciones: Esta variable es de suma importancia ya que influye en nuestro metodo tabu que indicara cuantas repeticiones o vueltas buscara una solucion quedandose al final con la mejor

tamaño: Esta variable se utiliza para guardar un número máximo de movimientos que el algoritmo recuerda para no repetirlos

semilla: Sirve para controlar la aleatoriedad y hacer que los resultados sean reproducibles. Nos ayuda mucho ya que podemos explorar mas posibles soluciones y que el programa no se quede estancado

df_distacias: En esta variable guardamos un dataframe utilizando la librería panda donde pasamos la tabla de excel

df_combustible: Lo mismo que con el dataframe de distancias pero aqui guardamos del excel de costo de combustible

Ejemplo:

	1	2	3	...
1	0	5	8	...
2	5	0	3	...
3	8	3	0	...

matriz_distancias: Convierte el dataframe de distancias a una matriz numerica para hacer mas manejable la matriz de esta forma

matriz_combustible: Convierte el dataframe de combustible a una matriz numerica para hacer mas manejable la matriz de esta forma

filas_d: numero de fila de la matriz distancia

columnas_d: numero de columna de la matriz distancia

filas_c: numero de fila de matriz combustible

filas_d: numero de columna de matriz combustible

asignacion_centros: Esta variable es una lista donde se le asigna las tiendas a cada centro correspondiente

df_asignacion: Es un dataframe con los detalles de la asignacion de las tiendas

Tienda	Centro_Asignado	Distancia	Costo
11	1	5	20
12	2	3	15

resumen_rutas: es una lista con los detalles finales de cada centro

Ejemplo:

[

{

"Centro": 1,

"Numero_Tiendas": 3,

"Distancia_Ruta": 120,

"Costo_Combustible_Ruta": 300,

"Ruta": "Nodo_1 -> Nodo_11 -> Nodo_25 -> Nodo_30 -> Nodo_1"

}

]

detalle_rutas: Es una lista con cada ruta para cada centro

solucion_actual: La ruta que se esta evaluando en ese momento del tabu

costo_actual: El costo del combustible de la ruta actual

mejor_solucion: La mejor ruta encontrada hasta el momento

mejor_costo_combustible: La ruta con el menor costo encontrado

lista_tabu: Lista de movimientos prohibidos por el momento en forma de tupla

vecinos: El conjunto de las rutas posibles cercanas a la actual

mejor_vecino: En esta variable se guarda el mejor candidato entre los vecinos

mejor_movimiento: El swap que se hizo entre los vecinos

Funciones

distancia(nodo_a, nodo_b)

Codigo:

```
def distancia(nodo_a, nodo_b):

    return matriz_distancias[nodo_a - 1][nodo_b - 1]
```

Esta funcion tiene como propósito consultar la distancia entre dos nodos dentro de la matriz de distancias. Su trabajo consiste en tomar dos nodos, por ejemplo un centro de distribución y una tienda, y devolver el valor numérico que representa la distancia entre ambos.

Ejemplo:

distancia(3, 15)

Te devuelve

matriz_distancias[2][14]

costo_combustible_tramo(nodo_a, nodo_b)

Codigo:

```
def costo_combustible_tramo(nodo_a, nodo_b):

    return matriz_combustible[nodo_a - 1][nodo_b - 1]
```

Realiza una tarea muy parecida a la anterior, pero en lugar de consultar distancias, consulta el costo de combustible entre dos nodos. Esta función toma dos nodos y busca en la matriz de costos el valor correspondiente al tramo que conecta esos puntos.

distancia_total_ruta(ruta)

Codigo

```
def distancia_total_ruta(ruta):

    if len(ruta) <= 1:

        return 0


    total = 0

    for i in range(len(ruta) - 1):

        total += distancia(ruta[i], ruta[i + 1])


    total += distancia(ruta[-1], ruta[0])

    return total
```

Esta funcion se encarga de calcular la distancia total de una ruta completa. Recibe una lista de nodos que representa el recorrido que sigue un centro de distribución al visitar sus tiendas asignadas. La función suma la distancia entre cada nodo consecutivo de la lista y, además, agrega el regreso del último nodo al primero para cerrar la ruta. Esto significa que no solo se evalúa el trayecto de salida, sino también el retorno al centro, lo cual es importante para representar de forma realista el recorrido total.

primero valida si la ruta tiene sentido

if len(ruta) <= 1:

return 0

Si la ruta solo tiene un nodo o está vacía, no hay recorrido real.

segundo suma tramo por tramo

for i in range(len(ruta) - 1):

total += distancia(ruta[i], ruta[i + 1])

Ejemplo:

[1, 12, 20, 30]

entonces suma:

- 1 → 12
- 12 → 20
- 20 → 30

costo_total_combustible_ruta(ruta)

Codigo:

```
def costo_total_combustible_ruta(ruta):

    if len(ruta) <= 1:

        return 0



    total = 0

    for i in range(len(ruta) - 1):

        total += costo_combustible_tramo(ruta[i], ruta[i + 1])



    total += costo_combustible_tramo(ruta[-1], ruta[0])

    return total
```

Esta funcion calcula el costo total de combustible de una ruta cerrada. Su lógica es muy similar a la función de distancia total, pero en lugar de sumar distancias, suma los costos de combustible entre cada tramo consecutivo de la ruta. También incluye el regreso al punto inicial, de forma que la ruta quede completa. Esta función es una de las más importantes de todo el programa, ya que el algoritmo de búsqueda tabú utiliza precisamente este valor para comparar rutas y determinar cuál es más conveniente.

generar_ruta_inicial(deposito, tiendas_asignadas)

Codigo:

```
def generar_ruta_inicial(deposito, tiendas_asignadas):

    ruta_tiendas = tiendas_asignadas[:]

    random.shuffle(ruta_tiendas)

    return [deposito] + ruta_tiendas
```

Esta función crea una primera solución para comenzar el proceso de optimización. Su trabajo es recibir el centro de distribución y la lista de tiendas asignadas a ese centro, mezclar el orden de las tiendas de manera aleatoria y devolver una lista que comience con el depósito seguido de las tiendas en ese orden mezclado. Esta ruta inicial no necesariamente es la mejor, pero sirve como punto de partida para que la búsqueda tabú empiece a explorar mejoras.

generar_vecinos_intercambio(ruta)

Codigo:

```
def generar_vecinos_intercambio(ruta):

    vecinos = []



    if len(ruta) <= 2:

        return vecinos



    for i in range(1, len(ruta)):

        for j in range(i + 1, len(ruta)):
```

```
            nueva_ruta = ruta[:]

            nueva_ruta[i], nueva_ruta[j] = nueva_ruta[j],
nueva_ruta[i]



            movimiento = tuple(sorted((ruta[i], ruta[j])))

            vecinos.append((nueva_ruta, movimiento))



    return vecinos
```

Tiene la tarea de producir soluciones vecinas a partir de una ruta actual. En la búsqueda tabú, una solución vecina es una alternativa muy parecida a la solución actual, pero con un cambio pequeño. En este caso, el cambio consiste en intercambiar la posición de dos tiendas dentro de la ruta. La función recorre todas las posibles parejas de posiciones, excepto la posición cero, ya que esa corresponde al centro de distribución y no debe moverse. Por cada intercambio posible, se genera una nueva ruta y se guarda también el movimiento que produjo ese cambio. Esto permite que el algoritmo tenga varias rutas candidatas para evaluar en cada iteración.

Ejemplo:

Si la ruta actual es:

[1, 11, 12, 13]

los vecinos posibles serían:

intercambiar 11 y 12 → `[1, 12, 11, 13]`

intercambiar 11 y 13 → `[1, 13, 12, 11]`

intercambiar 12 y 13 → `[1, 11, 13, 12]`

`imprimir_ruta(ruta)`

Codigo:

```
def imprimir_ruta(ruta):
```

```
    if len(ruta) == 1:
        return f"Nodo_{ruta[0]} -> Nodo_{ruta[0]}"

    ruta_cerrada = ruta + [ruta[0]]
    return " -> ".join(f"Nodo_{n}" for n in ruta_cerrada)
```

sirve para representar la ruta en un formato legible para el usuario. En lugar de mostrar la lista como un arreglo de números, la transforma en un texto con flechas que indican el recorrido entre nodos.

Ejemplo:

si la ruta es [1, 11, 20, 35]

imprime

Nodo_1 -> Nodo_11 -> Nodo_20 -> Nodo_35 -> Nodo_1

asignar_tiendas_a_centros(centros, tiendas)

Codigo:

```
def asignar_tiendas_a_centros(centros, tiendas):
    asignacion = {centro: [] for centro in centros}
    detalle = []

    for tienda in tiendas:
        mejor_centro = None
        mejor_distancia = float("inf")

        for centro in centros:
```

```
            d = distancia(centro, tienda)


            if d < mejor_distancia:

                mejor_distancia = d

                mejor_centro = centro


        asignacion[mejor_centro].append(tienda)


        detalle.append({

            "Tienda": tienda,

            "Centro_Asignado": mejor_centro,

            "Distancia_Centro_Tienda": mejor_distancia,

            "Costo_Combustible_Centro_Tienda":
costo_combustible_tramo(mejor_centro, tienda)

        })


    return asignacion, pd.DataFrame(detalle)
```

Esta funcion es la encargada de resolver la primera etapa del problema: determinar qué tiendas le corresponden a cada centro de distribución. Para hacerlo, revisa cada tienda una por una y calcula su distancia con respecto a todos los centros disponibles. Después compara esas distancias y elige el centro con menor distancia directa hacia esa tienda. Una vez identificado el mejor centro, la tienda se agrega a la lista de tiendas asignadas a ese depósito.

busqueda_tabu_combustible(deposito, tiendas_asignadas, iteraciones, tam_tabu, mostrar_progreso)

Codigo:

```
    if len(tiendas_asignadas) == 0:
```

```
        return [deposito], 0, 0


    if len(tiendas_asignadas) == 1:

        ruta = [deposito] + tiendas_asignadas

        return ruta, distancia_total_ruta(ruta),
costo_total_combustible_ruta(ruta)


    solucion_actual = generar_ruta_inicial(deposito,
tiendas_asignadas)

    costo_actual = costo_total_combustible_ruta(solucion_actual)


    mejor_solucion = solucion_actual[:]

    mejor_costo_combustible = costo_actual


    lista_tabu = []


    if mostrar_progreso:

        print(f"\n=== CENTRO {deposito} | RUTA INICIAL ===")

        print(imprimir_ruta(solucion_actual))

        print(f"Distancia inicial:
{distancia_total_ruta(solucion_actual):.4f}")

        print(f"Costo combustible inicial: {costo_actual:.4f}\n")


    for it in range(iteraciones):

        vecinos = generar_vecinos_intercambio(solucion_actual)


        mejor_vecino = None

        mejor_movimiento = None
```

```
        mejor_costo_vecino = float("inf")

        for vecino, movimiento in vecinos:
            costo_vecino = costo_total_combustible_ruta(vecino)

            es_tabu = movimiento in lista_tabu
            mejora_global = costo_vecino < mejor_costo_combustible

            if (not es_tabu) or mejora_global:
                if costo_vecino < mejor_costo_vecino:
                    mejor_vecino = vecino
                    mejor_movimiento = movimiento
                    mejor_costo_vecino = costo_vecino

        if mejor_vecino is None:
            continue

        solucion_actual = mejor_vecino[:]
        costo_actual = mejor_costo_vecino

        lista_tabu.append(mejor_movimiento)
        if len(lista_tabu) > tamaño:
            lista_tabu.pop(0)

        if costo_actual < mejor_costo_combustible:
            mejor_solucion = solucion_actual[:]
            mejor_costo_combustible = costo_actual
```

```
        if mostrar_progreso:
            print(
                f"Centro {deposito} | Iteración {it+1:03d} | "
                f"Costo combustible actual: {costo_actual:.4f} | "
                f"Mejor costo combustible:
{mejor_costo_combustible:.4f}"
            )


    return (
        mejor_solucion,
        distancia_total_ruta(mejor_solucion),
        costo_total_combustible_ruta(mejor_solucion)
    )
```

Es la función principal del programa y la que implementa el método de búsqueda tabú. Su objetivo es encontrar la mejor ruta posible para un centro de distribución, minimizando el costo total de combustible. Primero revisa si el centro no tiene tiendas asignadas o si solo tiene una, ya que en esos casos no hay realmente un problema complejo de optimización. Después genera una ruta inicial aleatoria y calcula su costo de combustible. Esa primera ruta se toma como la solución actual y también como la mejor conocida al inicio. A partir de ahí, el algoritmo entra en un ciclo de iteraciones donde, en cada paso, genera todas las soluciones vecinas posibles intercambiando tiendas de lugar. Luego evalúa el costo de combustible de cada vecino y selecciona el mejor de ellos, siempre verificando si el movimiento que lo generó no está prohibido por la lista tabú.

Conclusión

Después de realizar el trabajo podemos llegar a la conclusión que los métodos de búsqueda como es el algoritmo de búsqueda tabú que es el que nosotros utilizamos para nuestro proyecto, podemos ver como la tecnología marca realmente la diferencia en los costos de una empresa.

Por mas grande que sea un problema para resolver la inteligencia artificial nos permite encontrar las rutas mas efectivas en poco tiempo, todo esto gracias a la memoria que utiliza el algoritmo evitando repetir soluciones que no sirven

En resumen, las herramientas heurísticas no sirven cuando los datos son demasiado para los métodos tradicionales

Una buena implementación y diseño de las rutas puede llegar ahorrar dinero y también hace que las operaciones sean mas organizadas

Fuentes

Búsqueda tabú
Academia Lab. (s.f.). *Búsqueda tabú*. Recuperado el 30 de marzo de 2026, de https://academia-lab.com/enciclopedia/busqueda-tabu/

Recocido Simulado

Eurystic Solutions. (2024, noviembre 7). *Algoritmo de recocido simulado: Optimización inspirada en la física*. https://eurysticsolutions.com/es/2024/11/07/algoritmo-de-recocido-simulado-optimizacion-inspirada-en-la-fisica/